

Spark Broadcast Join Internals

Prepared By : Kaviraj T U

How Spark Avoids Expensive Shuffles

- 👉 One of the most powerful optimizations in Spark is the **Broadcast Join**.
 - 👉 When used correctly, it can turn a **very expensive shuffle join** into a **much faster local join**.
 - 👉 But when used incorrectly, it can cause **executor OOM failures**.
 - 👉 Understanding how broadcast join works internally is key for Spark performance tuning.
-

What Problem Does Broadcast Join Solve?

- 👉 Normally when Spark performs a join:
- 👉 Example:

```
Large Table A  (500GB)
Large Table B  (200GB)
```

- 👉 Spark must:
 - Shuffle both datasets
 - Repartition data

- Move records across executors

👉 This causes:

- Network overhead
- Disk spill
- Large shuffle stages

👉 Broadcast join avoids this.

How Broadcast Join Works

👉 Instead of shuffling both tables:

👉 Spark does this:

- ✓ Small table is **collected by the driver**
- ✓ Sent (broadcast) to all executors
- ✓ Stored in executor memory
- ✓ Large table partitions join locally with broadcasted data

👉 So only one dataset moves — not both.

👉 Result:

- ✓ No shuffle for large dataset
 - ✓ Faster join
 - ✓ Reduced network traffic
-

Example Scenario

Fact Table (1TB)

Dimension Table (5MB)

👉 Without broadcast:

Fact shuffle
Dimension shuffle
Join

👉 With broadcast:

Broadcast dimension
Local join on each executor

👉 Massive performance improvement.

How Spark Chooses Broadcast Join

👉 Spark automatically uses broadcast join if table size is below threshold.

👉 Default threshold:

```
spark.sql.autoBroadcastJoinThreshold
```

👉 Default value:

10MB

👉 If dataset is smaller than threshold → Spark may broadcast it.

Manual Broadcast Example

👉 Sometimes you know a table is small.

👉 You can force broadcast:

```
from pyspark.sql.functions import broadcast  
  
df_large.join(broadcast(df_small), "id")
```

👉 Spark will send `df_small` to all executors.

Common Broadcast Join Problems

✅ Broadcasting Large Tables

👉 If you broadcast a large dataset:

👉 Example:

```
Broadcast size = 500MB  
Executors = 20
```

👉 Memory required:

```
500MB × 20 = 10GB
```

👉 Executors may run out of memory.

👉 Result:

Executor OOM

✓ Driver Memory Pressure

- 👉 Broadcast data is first collected on the driver.
- 👉 Large broadcast datasets can overwhelm driver memory.

✓ Multiple Broadcast Joins

- 👉 If multiple joins broadcast different tables:
- ✓ Executors may store several broadcast variables simultaneously.
- ✓ This increases memory pressure.

How to Verify Broadcast Join

👉 In Spark UI:

👉 Go to:

SQL → Physical Plan

👉 You'll see:

BroadcastHashJoin

or

BroadcastExchange

👉 This confirms Spark used broadcast join.

Best Practices

- ✓ Broadcast small dimension tables
 - ✓ Avoid broadcasting datasets >100MB (general rule)
 - ✓ Monitor executor memory usage
 - ✓ Use broadcast hints carefully
 - ✓ Combine with AQE for smarter join selection
 - 👉 Broadcast join is powerful — but not free.
-

Final Takeaway

- 👉 Broadcast join is one of Spark's smartest optimizations.
 - 👉 It replaces expensive shuffle joins with fast local joins.
 - 👉 But it works best when:
 - One table is small
 - Executor memory is sufficient
 - Broadcast size is controlled
 - 👉 When used correctly, it can reduce job runtime dramatically.
 - 👉 Understanding join strategies is a key step toward becoming a **Spark performance engineer**.
-

Thank you.!

Happy Learning...